EXP NO: 4B	*ENSEMBLE METHODS: RANDOM FOREST*

AIM:

To implement the Random Forest ensemble method for classification on the Breast Cancer dataset, compare it with a single Decision Tree, study the effect of the number of trees, and evaluate the model using accuracy, out-of-bag score, confusion matrix, ROC-AUC, cross-validation and feature importance.

ALGORITHM:

1. Import the required libraries (numpy, pandas, matplotlib, seaborn, scikit-learn).
2. Load the Breast Cancer dataset (30 features, binary classes: malignant and benign).
3. Split the dataset into training (80%) and testing (20%) sets using stratified sampling.
4. Train a single Decision Tree as the baseline model and find its test accuracy.
5. Build the Random Forest using RandomForestClassifier() with 100 trees and the out-of-bag (OOB) score enabled, and train it using fit().
6. Predict the test data and evaluate using accuracy, OOB score, ROC-AUC, confusion matrix and classification report.
7. Compare the Decision Tree and Random Forest using 5-fold cross-validation.
8. Study the effect of the number of trees (n_estimators) on test accuracy.
9. Find the feature importances and tune the hyperparameters using GridSearchCV.
10. Plot the confusion matrix, accuracy versus number of trees, and the top feature importances.

CODE:

```
# 1. Import libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, cross_val_score,

    GridSearchCV
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix,
```

```
    classification_report, roc_auc_score

# 2. Load the dataset (Breast Cancer - binary classification)
data = load_breast_cancer()
X, y = data.data, data.target
print("Dataset shape:", X.shape)
print("Classes:", ", ".join(data.target_names))

# 3. Split into training and testing sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# 4. Baseline: a single Decision Tree
dt = DecisionTreeClassifier(random_state=42)
dt.fit(X_train, y_train)
dt_acc = accuracy_score(y_test, dt.predict(X_test))
print("\nSingle Decision Tree Accuracy:", round(dt_acc, 4))

# 5. Ensemble method: Random Forest (100 trees, out-of-bag score enabled)
rf = RandomForestClassifier(n_estimators=100, oob_score=True, random_state=42)
rf.fit(X_train, y_train)

# 6. Predict and evaluate
y_pred = rf.predict(X_test)
print("Random Forest Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Random Forest OOB Score:", round(rf.oob_score_, 4))
print("Random Forest ROC-AUC :", round(roc_auc_score(y_test,
    rf.predict_proba(X_test)[:, 1]), 4))
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n",
      classification_report(y_test, y_pred, target_names=data.target_names))

# 7. 5-fold cross-validation: Decision Tree vs Random Forest
cv_dt = cross_val_score(DecisionTreeClassifier(random_state=42), X, y, cv=5)
cv_rf = cross_val_score(RandomForestClassifier(n_estimators=100,
    random_state=42), X, y, cv=5)
print("Cross-validation accuracy (mean +/- std):")
print(f" Decision Tree : {cv_dt.mean():.4f} +/- {cv_dt.std():.4f}")
print(f" Random Forest : {cv_rf.mean():.4f} +/- {cv_rf.std():.4f}")

# 8. Effect of the number of trees
tree_counts = [1, 5, 10, 25, 50, 100, 200]
scores = []
for n in tree_counts:
    m = RandomForestClassifier(n_estimators=n, random_state=42).fit(X_train,
        y_train)
    scores.append(accuracy_score(y_test, m.predict(X_test)))
    print(f"n_estimators = {n:3d} -> Test Accuracy = {scores[-1]:.4f}")

# 9. Feature importance
importance = pd.Series(rf.feature_importances_,
    index=data.feature_names).sort_values(ascending=False)
print("\nTop 10 Important Features:\n", importance.head(10).round(4))

# 10. Hyperparameter tuning with GridSearchCV
```

```
param_grid = {'n_estimators': [100, 200], 'max_depth': [None, 5, 10],
              'min_samples_split': [2, 5], 'max_features': ['sqrt', 'log2']}
grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5,
    n_jobs=-1)
grid.fit(X_train, y_train)
best_acc = accuracy_score(y_test, grid.best_estimator_.predict(X_test))
print("\nBest Parameters:", grid.best_params_)
print("Best CV Accuracy:", round(grid.best_score_, 4))
print("Tuned Random Forest Test Accuracy:", round(best_acc, 4))

# 11. Visualisation
fig, ax = plt.subplots(1, 2, figsize=(12, 4.5))
sns.heatmap(confusion_matrix(y_test, y_pred), annot=True, fmt='d',
    cmap='Greens', ax=ax[0],
            xticklabels=data.target_names, yticklabels=data.target_names)
ax[0].set_title("Random Forest Confusion Matrix");
    ax[0].set_xlabel("Predicted"); ax[0].set_ylabel("Actual")
ax[1].plot(tree_counts, scores, 'o-', color='darkgreen')
ax[1].axhline(dt_acc, color='red', linestyle='--', label=f"Single Decision Tree
    ({dt_acc:.3f})")
ax[1].set_xlabel("Number of trees"); ax[1].set_ylabel("Test Accuracy")
ax[1].set_title("Effect of Number of Trees"); ax[1].legend()
plt.tight_layout()
plt.show()

plt.figure(figsize=(8, 4.5))
importance.head(10).sort_values().plot(kind='barh', color='teal')
plt.title("Top 10 Feature Importances (Random Forest)");
    plt.xlabel("Importance")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
Dataset shape: (569, 30)
Classes: malignant, benign


Single Decision Tree Accuracy: 0.9123
Random   Forest   Accuracy   :   0.9561
Random Forest OOB Score: 0.9538
Random Forest ROC-AUC  : 0.9937

Confusion Matrix:
 [[39  3]
 [ 2 70]]

Classification Report:
              precision    recall  f1-score   support

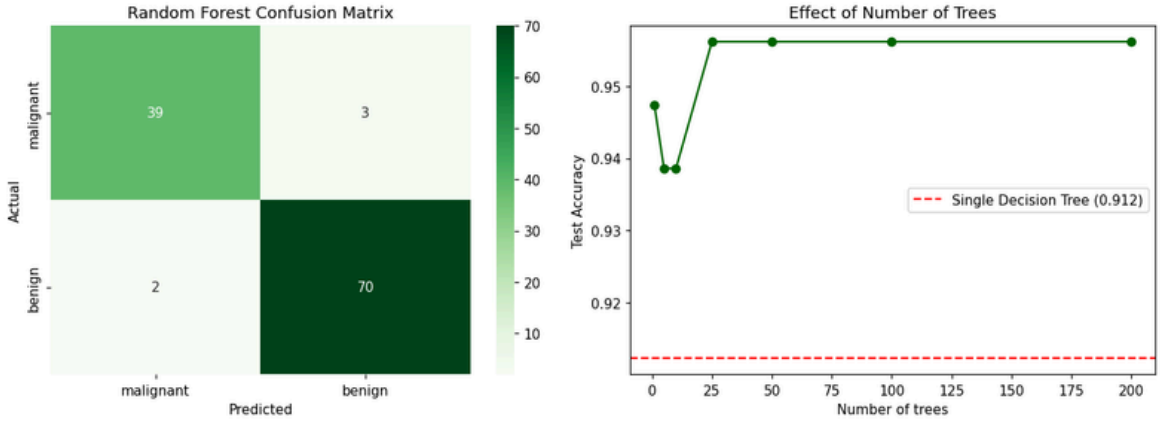
   malignant       0.95      0.93      0.94        42
      benign       0.96      0.97      0.97        72

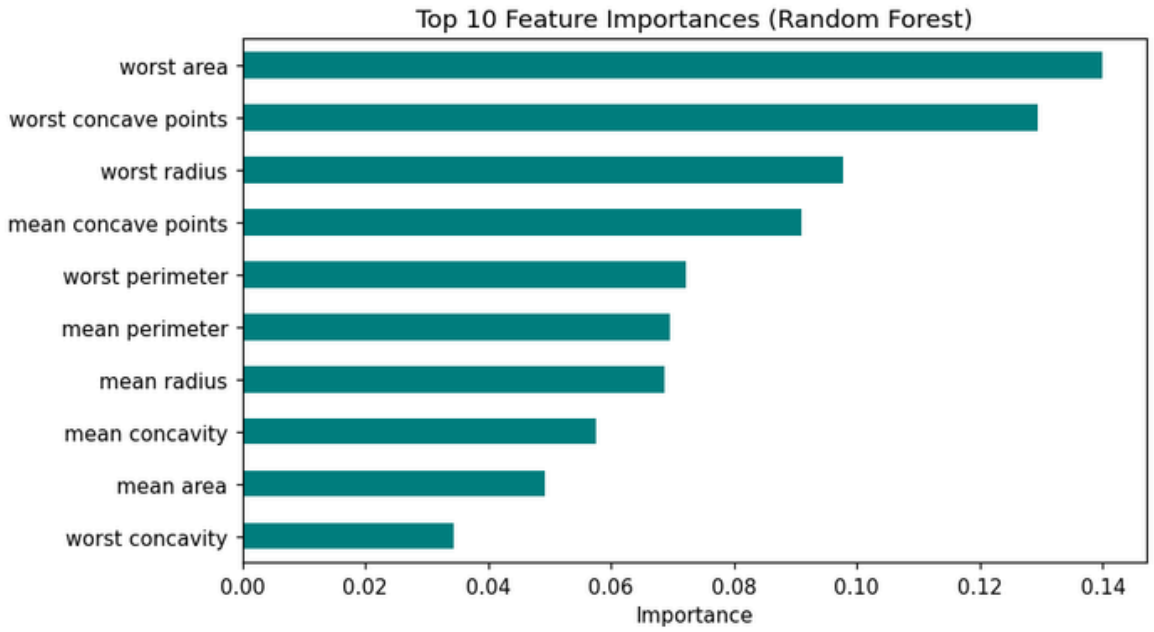
    accuracy                           0.96       114
macro    avg       0.96      0.95      0.95       114
weighted avg       0.96      0.96      0.96       114

Cross-validation accuracy (mean +/- std):
  Decision Tree : 0.9173 +/- 0.0242
  Random Forest : 0.9561 +/- 0.0228
n_estimators     1 = ->  Test Accuracy = 0.9474
n_estimators     5 = ->  Test Accuracy = 0.9386
n_estimators    10 = ->  Test Accuracy = 0.9386
n_estimators    25 = ->  Test Accuracy = 0.9561
n_estimators    50 = ->  Test Accuracy = 0.9561
n_estimators  =  100 ->  Test Accuracy = 0.9561
n_estimators = 200  ->  Test Accuracy = 0.9561


Top 10 Important Features:
 worst area              0.1400
worst concave points    0.1295
worst radius            0.0977
mean concave points     0.0909
worst perimeter         0.0722
mean perimeter          0.0696
mean radius             0.0687
mean concavity          0.0576
mean area               0.0492
worst concavity         0.0343
dtype: float64


Best Parameters: {'max_depth': None, 'max_features': 'sqrt',
    'min_samples_split': 2, 'n_estimators': 200}
Best CV Accuracy: 0.9604
Tuned Random Forest Test Accuracy: 0.9561
```





RESULT:

The Random Forest ensemble was successfully implemented on the Breast Cancer dataset. It achieved a test accuracy of 95.61% (OOB score 95.38%, ROC-AUC 0.9937), higher than the 91.23% of a single Decision Tree, and the 5-fold cross-validation accuracy was also higher (95.61% against 91.73%). The accuracy became stable from 25 trees onwards, and worst area, worst concave points and worst radius were the most important features. Hyperparameter tuning raised the cross-validation accuracy to 96.04% while the test accuracy stayed at 95.61%.